

RL Programming Assignment 1

Marumamula Venkata Pranay (23B1073)

September 2025

1 Task 1

- **UCB** : Start by pulling each arm once. Then calculate the UCB value for each arm as the sum of empirical mean, and the exploration bonus.

$$ucb_a^t = \hat{p}_a^t + \sqrt{\frac{2 \cdot \log(t)}{u_a^t}}$$

where u_a^t represents the number of times arm a was pulled. Function `get_reward()` updates the number of successes, pulls, and the empirical mean of an arm. Function `give_pull()` returns the index of the arm with the maximum value of `ucb`.

- **KL-UCB** : Start by pulling each arm once. The KL-UCB value of an arm is defined as

$$kl_ucb_a^t = \max_q [KL(p, q) \leq \frac{\log(t) + C \cdot \log(\log(t))}{u_a^t}]$$

where KL is the KL divergence function defined between two Bernoulli distributions with means p, q as

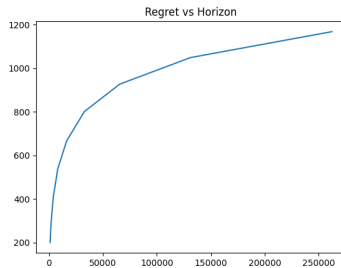
$$KL(p, q) = p \cdot \log\left(\frac{p}{q}\right) + (1 - p) \cdot \log\left(\frac{1 - p}{1 - q}\right)$$

Since the KL function is monotonically increasing, we can binary search for the value of q , and thereby $kl_ucb_a^t$. The `give_pull()` function returns the index of the arm with the maximum value of `kl-ucb`.

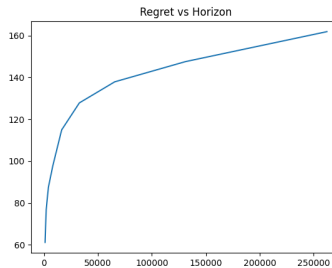
- **Thompson Sampling** Thompson sampling models the mean of an arm as

$$\mu_a^t = \beta(s + 1, f + 1)$$

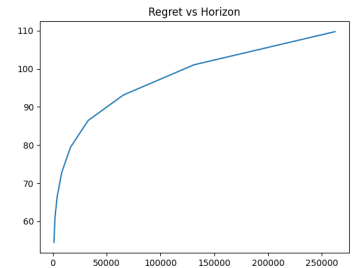
where s and f represent the number of successful and failed pulls of the arm until time t . Very straightforward to implement.



(a) UCB



(b) KL UCB



(c) Thompson Sampling

2 Task 2

Let us first understand how this is a different optimization problem as compared to the previous task. UCB, KL-UCB, and Thompson sampling achieve the best results when trying to minimize long term regret. That is, $R(T) \in O(\log(T))$ as $T \rightarrow \infty$. In this case, we want to find a good arm, and exploit it continuously to obtain some reward in the minimum number of steps.

- Pull the current arm in consideration P times.
- If the empirical mean achieved is above some threshold T , we can declare this arm as a "satisfactory" arm, and keep exploiting until it breaks.
- Else, move on to the next arm, and repeat.
- If no arm was found to be "satisfactory", declare the best arm as the arm with the highest empirical mean. Commit to this arm, and pull it until it breaks.

This simple greedy approach performs very well compared to algorithms like KL-UCB (adjusted for poisson rewards) after some tuning on the parameters P and T .

3 Task 3

3.1 Batch-Optimized KL-UCB

The standard KL-UCB algorithm updates the upper confidence bound of every arm at every time step. This gives good theoretical guarantees, but the per-step computation is expensive since it requires a binary search for each arm.

The optimized version follows the same principle but avoids recomputing at every round. The idea is:

1. Compute KL-UCB values for all arms.
2. Find the arm with the highest UCB (a^*) and the second highest arm (a').
3. Assume a worst case for a^* where all future rewards are 0. Under this assumption, calculate how many times in a row we can pull a^* before its UCB drops below that of a' .
4. This number of *safe pulls* is found by binary search. (We can do this because it's possible until a certain value, and after, it's not possible)
5. Pull a^* that many times without updating the UCBs in between.

This method assumes that the numerator $\log(t) + C \cdot \log(\log(t))$ doesn't change much with time. Core logic :

$$KL(p_{worst}, q') \leq \frac{\log(t) + C \cdot \log(\log(t))}{n + \text{safe_pulls}}$$

where $p_{worst} = \frac{p^* n}{n + \text{safe_pulls}}$ and q' is the KL-UCB value of the second best arm.

Where it helps.

- We skip repeated KL computations when the same arm is clearly best.
- Regret remains (approximately) the same, since we only commit to pulls that are guaranteed not to change the leader.
- The method scales better for large horizons, since recomputation happens once per batch instead of every step.

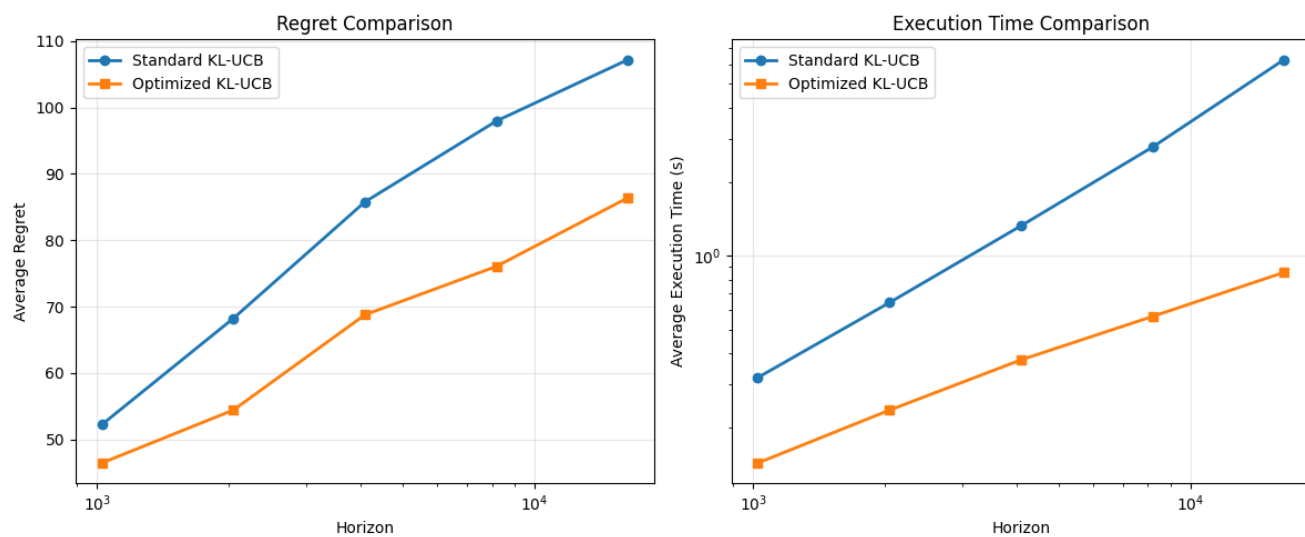


Figure 2: Task 3 Plot